

3E Tag Manager Deployment Guide

Version: 1.0.0

Last Updated: 2025-12-17

Author: 3E Enrollment Development Team

Table of Contents

1. [Introduction](#)
 2. [Prerequisites](#)
 3. [Base Solutions](#)
 4. [Chatbot Solutions](#)
 5. [Pop-up Solutions](#)
 6. [General Troubleshooting](#)
 7. [Debug Mode Usage](#)
-

Introduction

This guide provides step-by-step instructions for deploying and testing 3E Tag Manager scripts in Google Tag Manager (GTM). Think of GTM as a container that holds all your website tracking scripts. Each tag (script) is documented with: - **What it does** - In plain English, what problem it solves - **How it works** - The step-by-step process it follows - **How to test it** - Exact steps to verify it's working - **How to fix problems** - Common issues and solutions

Who This Guide Is For: - Anyone deploying tags to websites - People testing website functionality - Support staff helping troubleshoot issues - Anyone who needs to understand what these tags do

What This Guide Covers: - Where to find each tag file - What each tag does (explained simply) - How each tag works (step-by-step process) - How to test each tag (validation procedures) - How to fix common problems (troubleshooting)

Important Terms Explained: - **GTM (Google Tag Manager):** A tool that manages all your website tracking scripts in one place - **Tag:** A piece of code (script) that does something on

your website (like tracking clicks or loading a chatbot) - **dataLayer**: A special list that stores events and data that tags can read and use - **Marketo Munchkin**: A tracking script from Marketo that records visitor activity - **Marketo Forms2**: The system that handles Marketo form submissions - **Console**: A developer tool in your browser (press F12) that shows messages and errors

Prerequisites

Required Setup

Before deploying any tags, ensure the following are configured:

1. **3E Config Variable Template**
2. **Location:** `tags/base-solutions/Template - 3E Config.html`
3. **Type:** GTM Variable Template
4. **Must be deployed FIRST** before any other tags
5. Contains all configuration settings used by other tags
6. **GTM Container Access**
7. Ensure you have appropriate permissions (Edit, Publish)
8. Verify container ID is correct
9. Confirm account ID matches the container
10. **Dependencies**
11. **Marketo Munchkin:** Required for tracking tags (loaded automatically or via 3E_Insights Pixel)
12. **Marketo Forms2:** Required for form-related tags (loaded automatically by Marketo forms)
13. **3E Config:** Required for all tags except Template - 3E Config

Debug Mode

Debug mode shows you detailed messages about what tags are doing. This is essential for testing and troubleshooting.

How to Enable: - Set `debugMode: 'true'` in 3E Config (use the word 'true' in quotes, not a checkbox) - Open your browser's developer console (press F12, then click the "Console" tab) - Reload the page - You'll see messages from each tag showing what it's doing

What You'll See: - Each tag's name and version number - Messages when tags start working - Messages when events happen (like form submissions or clicks) - Error messages if something goes wrong

Important: Always disable debug mode (`debugMode: 'false'`) before going live on your website, as it can slow down the page and expose internal details.

Base Solutions

Base solutions provide core functionality for all client implementations. These should be deployed for every client.

Template - 3E Config

File Location: `tags/base-solutions/Template - 3E Config.html`

Version: 2.2

Type: GTM Variable Template

Dependencies: None

What It Does

The 3E Config is a central configuration hub that stores all settings used by other tags. Think of it as a settings file that all other scripts read from. It must be set up as a GTM Variable Template before deploying any other tags.

How It Works

Think of 3E Config like a master settings file that all other tags read from. Here's the process:

1. **Setup:** You create the 3E Config as a Variable in GTM (like creating a settings file)
2. **Configuration:** You enter all your settings in one place (Marketo ID, chatbot settings, etc.)
3. **Access:** When other tags load, they automatically read these settings using `{{3E config}}`

4. **Benefits:** Instead of entering the same settings in every tag, you enter them once in 3E Config

Example Settings Stored: - Marketo tracking ID (munchkinId) - Marketo website URL (baseUrl) - Chatbot ID (for chatbot features) - Form validation settings (on/off switches) - Debug mode (on/off switch)

Validation Process

1. Verify Variable Setup:

2. Go to GTM → Variables
3. Confirm "3E Config" variable exists
4. Check that it's configured with required settings

5. Verify Configuration Values:

6. Open variable configuration
7. Verify `baseUrl` is set (Marketo instance URL)
8. Verify `munchkinId` is set (Marketo tracking ID)
9. Verify `debugMode` is set (for testing: 'true', for production: 'false')

10. Test Variable Access:

11. Enable debug mode in 3E Config (set `debugMode: 'true'`)
12. Deploy a tag that uses 3E Config (e.g., 3E_Analytics Tracking)
13. Open browser console (press F12, click "Console" tab)
14. Reload the page
15. Look for initialization messages from the tag
16. Verify there are NO "Failed to load 3E config" error messages
17. If you see errors, the variable isn't set up correctly

Troubleshooting

Issue: Tags show "Failed to load 3E config" errors

Solution: - **Check Variable Exists:** Go to GTM → Variables → Look for "3E Config" variable -

Check Variable Name: The name must be exactly "3E Config" (case-sensitive, with space and capital E) - **Check Variable Type:** It should be a Variable Template (not a regular variable)

- **Check if Published:** Make sure the variable is published in your GTM container (not just

saved as a draft) - **Check Variable Reference:** In your tags, make sure you're using `{{3E config}}` (with space, case-sensitive)

Issue: Configuration values not applying

Solution: - **Check if Published:** Make sure you've published the variable in GTM (not just saved as draft) - **Check for Typos:** Look for spelling mistakes in configuration keys (like

`munchkinId` vs `munchinId`) - **Check JSON Format:** Make sure your configuration is valid

JSON (proper quotes, commas, brackets) - **Clear Cache:** Clear your browser cache

(Ctrl+Shift+Delete or Cmd+Shift+Delete) and reload the page - **Check Debug Mode:** Enable debug mode and check console to see what values are actually being read

3E_Analytics Tracking

File Location: `tags/base-solutions/3E_Analytics Tracking.html`

Version: 1.2.0

Type: Custom HTML Tag

Dependencies: 3E Config

What It Does

Tracks user interactions on the page including button clicks, scroll depth, form interactions, video engagement, and page performance. This data is sent to analytics platforms and helps understand how users engage with the website.

How It Works

This script watches what users do on your website and records it. Here's the process:

1. **Page Load:** When someone visits your page, the script starts running
2. **Watching:** It continuously watches for user actions:
 3. Button clicks
 4. How far they scroll down the page
 5. Form interactions (typing, clicking fields)
 6. Video watching (play, pause, completion)
 7. How fast the page loads
8. **Recording:** When something happens, it records it:
 9. Scroll depth: Records when user scrolls 25%, 50%, 75%, or 100% down the page
 10. Clicks: Records which buttons/links were clicked
 11. Forms: Records when users interact with form fields

12. Videos: Records when videos are played, paused, or completed
13. **Sending Data:** It sends this information to the dataLayer (a special list that other tools can read)

Why This Matters: This data helps you understand how people use your website - what they click, how much they read, and what interests them.

Validation Process

1. **Enable Debug Mode:**
2. Set `debugMode: 'true'` in 3E Config
3. Reload page
4. **Check Console Logs:**
5. Open browser console (F12)
6. Look for: `[3E_Analytics Tracking] v1.2.0`
7. Verify script initialized message
8. **Test Scroll Tracking:**
9. Scroll page to 25%, 50%, 75%, 100%
10. Check console for scroll depth events
11. Verify dataLayer events: `scroll_depth_25` , `scroll_depth_50` , etc.
12. **Test Click Tracking:**
13. Click on buttons/links with CTA classes
14. Check console for click tracking messages
15. Verify dataLayer events for clicks
16. **Test Form Interaction:**
17. Interact with form fields
18. Check console for form interaction events
19. Verify dataLayer events for form interactions

Troubleshooting

Issue: No scroll tracking events

Solution: - **Check Script Loaded:** Open console (F12) and look for [3E_Analytics Tracking] v1.2.0 message - **Check Page Length:** The page needs to be tall enough to scroll. If the page fits on one screen, scroll events won't fire - **Check 3E Config:** Verify 3E Config is loaded (no "Failed to load 3E config" errors) - **Check Debug Mode:** Make sure debug mode is enabled to see scroll event messages - **Try Scrolling:** Manually scroll to 25%, 50%, 75%, and 100% of the page and watch console for messages

Issue: Click events not firing

Solution: - **Check CSS Classes:** Verify buttons/links have the correct CSS classes that the script is looking for (check the script code or documentation for which classes it tracks) - **Check Console:** Enable debug mode and look for click detection messages when you click - **Check Element Visibility:** Make sure the button/link is actually clickable (not hidden behind another element or disabled) - **Check Browser Console:** Look for any JavaScript errors that might be preventing click tracking - **Test Different Elements:** Try clicking different buttons to see if some work and others don't

3E_Page Activity

File Location: tags/base-solutions/3E_Page Activity.html

Version: 1.3.1

Type: Custom HTML Tag

Dependencies: 3E Config, Marketo Munchkin

What It Does

Tracks how long users stay on a page by sending "visit" events to Marketo at regular intervals. This helps measure engagement and time spent on pages.

How It Works

This script measures how long visitors stay on your page by sending "I'm still here" messages to Marketo:

1. **Page Load:** When someone visits your page, the script starts
2. **Wait for Marketo:** It waits for Marketo's tracking system (Munchkin) to be ready
3. **Send Updates:** Every 30 seconds (or whatever interval you set), it sends a "visit" event to Marketo saying "This person is still on the page"

4. **Keep Tracking:** It continues sending these updates as long as the person stays on the page
5. **Stop When They Leave:** When the person navigates to another page or closes the browser, it stops

Why This Matters: This helps you measure engagement - how long people actually spend reading your content, not just how many people visited.

Validation Process

1. **Enable Debug Mode:**
2. Set `debugMode: 'true'` in 3E Config
3. Reload page
4. **Check Console Logs:**
5. Open browser console (F12)
6. Look for: `[3E_Page Activity] v1.3.1`
7. Verify "Page Activity Script Initialized" message
8. **Verify Marketo Munchkin:**
9. Check console for "Marketo Munchkin detected" message
10. If not detected, verify Munchkin is loaded or 3E_Insights Pixel is deployed
11. **Test Visit Tracking:**
12. Stay on page for 30+ seconds
13. Check console for visit event messages
14. Verify Marketo receives visit events (check Marketo activity log)

Troubleshooting

Issue: "Marketo Munchkin not detected" error

Solution: - **Check if Munchkin is Loaded:** Open browser console and type

`window.Munchkin` - if it says "undefined", Munchkin isn't loaded - **Load Munchkin:** Deploy

the 3E_Insights Pixel tag, which will load Munchkin if it's not already on the page - **Check**

Munchkin ID: Verify the `munchkinId` in 3E Config matches your Marketo instance ID (found in Marketo admin settings) - **Check Network Tab:** Open browser DevTools → Network tab →

Reload page → Look for requests to Marketo (should see munchkin.js loading) - **Check**

Timing: Sometimes Munchkin loads slowly - wait a few seconds and check again

Issue: Visit events not sending

Solution: - **Check Munchkin:** Verify Munchkin is loaded (type `window.Munchkin` in console - should not be undefined) - **Check Interval:** Verify the interval setting in 3E Config (default is 30 seconds, so you need to wait at least 30 seconds) - **Check Page Active:** Make sure the browser tab is active (not in background) - most browsers pause scripts in background tabs -

Check Debug Mode: Enable debug mode and watch console for visit event messages -

Check Marketo: Log into Marketo and check the activity log for the lead to see if visit events are being received

3E_Form Validation

File Location: `tags/base-solutions/3E_Form Validation.html`

Version: 7.3.1

Type: Custom HTML Tag

Dependencies: 3E Config

What It Does

Adds extra validation to Marketo forms to prevent spam and ensure data quality. It can check for duplicate names, block default names like "first" and "last", validate .edu email addresses, block specific IP addresses, and add math question challenges.

How It Works

This script adds extra security checks to your forms to prevent spam and bad data. Here's how:

1. **Wait for Forms:** The script waits for Marketo's form system to load
2. **Find Forms:** It looks for all Marketo forms on the page
3. **Skip Hidden Forms:** It ignores hidden forms used by chatbots (those are handled separately)
4. **Add Security Checks:** Based on your settings, it adds these checks:
5. **Name Check:** Makes sure first name and last name are different (prevents "John John")
6. **Default Name Block:** Blocks common test names like "first", "last", "test"
7. **Email Check:** Can require .edu email addresses only (for student forms)
8. **IP Blocking:** Can block specific IP addresses (for known spammers)

9. **Math Question:** Can add a simple math question (like "What is 2 + 3?") to prove the user is human
10. **Show Errors:** If any check fails, it shows an error message to the user
11. **Block Submission:** The form won't submit until all checks pass

Why This Matters: This prevents spam submissions, fake data, and ensures you only get legitimate form submissions from real people.

Validation Process

1. **Enable Debug Mode:**
2. Set `debugMode: 'true'` in 3E Config
3. Reload page with Marketo form
4. **Check Console Logs:**
5. Open browser console (F12)
6. Look for: `[3E_Form Validation] v7.3.1`
7. Verify "Form Validation Script Initialized" message
8. **Test Name Validation (if enabled):**
9. Set first name and last name to the same value
10. Try to submit form
11. Verify error message: "First Name and Last Name cannot be the same"
12. Form should not submit
13. **Test Default Name Prevention (if enabled):**
14. Enter "first" in first name field
15. Try to submit form
16. Verify error message about containing "first"
17. Form should not submit
18. **Test EDU Email Validation (if enabled):**
19. Enter non-.edu email address
20. Try to submit form
21. Verify error message: "Email must be a .edu address"

22. Form should not submit

23. Test Math Question (if enabled):

24. Verify math question field appears on form

25. Select wrong answer

26. Try to submit form

27. Verify error message about math question

28. Select correct answer

29. Form should submit successfully

Troubleshooting

Issue: Validation not running

Solution: - **Check Marketo Forms2:** Open console and type `window.MktoForms2` - if undefined, Forms2 isn't loaded - **Check Console Messages:** Enable debug mode and look for form detection messages when page loads - **Check Form Type:** Verify the form is not a hidden conversion form (those are intentionally skipped by this script) - **Check Form Loaded:** Make sure the Marketo form has actually loaded on the page (you should see it visually) - **Check Timing:** Sometimes forms load slowly - wait a few seconds and check console again

Issue: Math question not appearing

Solution: - **Check Setting:** Verify `formMathValidation: 'true'` is set in 3E Config (must be the string 'true', not a boolean) - **Check Console:** Enable debug mode and look for math question creation messages - **Check Form Space:** Verify the form has room for an additional field (some forms are too compact) - **Check Form Type:** Make sure this is a visible form (not a hidden conversion form) - **Check Form Loaded:** Ensure the Marketo form has fully loaded before the validation script runs

Issue: Validation blocking legitimate submissions

Solution: - **Check Settings:** Review all validation toggles in 3E Config - you may have enabled a validation that's too strict - **Check IP Blocking:** If IP blocking is enabled, verify your IP blocking list doesn't include legitimate IP addresses - **Test Different Inputs:** Try submitting with different email addresses and names to see which validation is blocking - **Check Error Messages:** Read the error message shown to the user - it will tell you which validation failed - **Disable Temporarily:** If needed, temporarily disable specific validations in 3E Config to test which one is causing the issue

3E_RFI Submit

File Location: tags/base-solutions/3E_RFI_Submit.html

Version: 1.4.5

Type: Custom HTML Tag

Dependencies: 3E Config, Marketo Forms2

What It Does

Handles form submissions from Marketo forms and 123FormBuilder forms. When a form is submitted successfully, it pushes an `rfi_submission` event to the dataLayer, which can trigger other tags or tracking. It also manages redirects to thank you pages.

Important: This tag skips hidden conversion forms (chatbot blind forms) to prevent duplicate events. Hidden forms are handled by the 3E_3EI Recruiter Conversion script.

How It Works

For Marketo Forms: 1. **Wait for Forms:** Script waits for Marketo's form system to load 2. **Find Forms:** It looks for Marketo forms on the page 3. **Check Form Type:** It checks if the form is a hidden chatbot form (if yes, it skips it - those are handled by a different script) 4. **Listen for Submission:** It watches for when someone successfully submits the form 5. **When Form Submits:** - Sends an `rfi_submission` event to the dataLayer (so other tools know a form was submitted) - Handles the redirect to the thank you page - Prevents Marketo's default redirect (so we can control where they go)

For 123FormBuilder Forms: 1. **Find Forms:** Script looks for 123FormBuilder forms (they have IDs starting with `cf_`) 2. **Watch for Success:** It listens for signals that the form was submitted successfully 3. **Multiple Detection Methods:** It uses several ways to detect submission: - Watches for success messages from the form - Monitors iframe changes (123FormBuilder uses iframes) - Listens for special messages (postMessage) from the form 4. **When Form Submits:** - Detects the submission through one of these methods - Sends an `rfi_submission` event to the dataLayer

Why This Matters: This ensures that every form submission is properly tracked and recorded, regardless of which form system you're using.

Validation Process

Scenario 1: Visible Marketo Form Submission

1. **Enable Debug Mode:**
2. Set `debugMode: 'true'` in 3E Config

3. Reload page with Marketo form

4. Check Console Logs:

5. Open browser console (F12)

6. Look for: `[3E_RFI Submit] v1.4.5`

7. Verify "Marketo form detected and ready" message

8. Submit Form:

9. Fill out Marketo form completely

10. Submit form

11. Check console for: `[3E_RFI Submit] Pushed rfi_submission event to dataLayer`

12. Verify dataLayer Event:

13. Open browser console (press F12, click "Console" tab)

14. Type: `dataLayer` and press Enter

15. You'll see an array of events

16. Scroll through the array and look for an object that contains: `{event: 'rfi_submission'}`

17. If you find it, the event was successfully pushed

18. **Tip:** You can also type `dataLayer.filter(e => e.event === 'rfi_submission')` to find just the rfi_submission events

19. Verify Redirect:

20. Check that page redirects to thank you page

21. Verify redirect happens correctly

Scenario 2: 123FormBuilder Form Submission

1. Enable Debug Mode:

2. Set `debugMode: 'true'` in 3E Config

3. Add `?3e_rfi_debug=true` to URL for enhanced logging

4. Reload page with 123FormBuilder form

5. Check Console Logs:

6. Open browser console (F12)

7. Look for: `[3E_RFI Submit] v1.4.5`

8. Verify "Found X potential 123FormBuilder form(s)" message

9. Submit Form:

10. Fill out 123FormBuilder form completely

11. Submit form

12. Check console for `scrollToTop postMessage` detection or URL change detection

13. Look for: `[3E_RFI Submit] Received scrollToTop postMessage` or `[3E_RFI Submit] Thank you page pattern detected`

14. Verify dataLayer Event:

15. Open browser console (press F12, click "Console" tab)

16. Type: `dataLayer` and press Enter

17. You'll see an array of events

18. Scroll through the array and look for an object that contains: `{event: 'rfi_submission'}`

19. If you find it, the event was successfully pushed

20. **Tip:** You can also type `dataLayer.filter(e => e.event === 'rfi_submission')` to find just the `rfi_submission` events

Scenario 3: Chatbot Blind Form Submission (Should Be Skipped)

1. Enable Debug Mode:

2. Set `debugMode: 'true'` in 3E Config

3. Reload page with chatbot

4. Check Console Logs:

5. Open browser console (F12)

6. Look for: `[3E_RFI Submit] v1.4.5`

7. Verify "Marketo form detected and ready" message

8. Submit Chatbot Blind Form:

9. Interact with chatbot

10. Submit blind form through chatbot

11. Check console for: `[3E_RFI Submit] Skipping hidden conversion form - handled by chatbot conversion script`
12. Verify 3E_RFI Submit does NOT push `rfi_submission` event
13. **Verify Only One Event:**
14. Check `dataLayer` for `rfi_submission` events
15. Should see only ONE event (from 3E_3EI Recruiter Conversion script)
16. Should NOT see duplicate events

Troubleshooting

Issue: Marketo form submission not triggering event

Solution: - **Check Forms2:** Verify Marketo Forms2 is loaded (type `window.MktoForms2` in console) - **Check Console:** Enable debug mode and look for form detection messages - **Check Form Type:** Verify the form is not a hidden conversion form (those are handled by a different script) - **Check for Conflicts:** Look for hardcoded RFI submit scripts on the page that might be interfering (check page source) - **Check Form Submission:** Make sure the form actually submits successfully (you should see a success message or redirect) - **Check dataLayer:** After submission, check `dataLayer` for the `rfi_submission` event

Issue: 123FormBuilder form not detected

Solution: - **Check Form ID:** Verify the form has an ID that starts with `cf_` (inspect the form element to see its ID) - **Check Console:** Enable debug mode and look for form detection messages (should say "Found X potential 123FormBuilder form(s)") - **Check Timing:** Verify the form is loaded before the script runs (forms in iframes sometimes load slowly) - **Enhanced Debugging:** Add `?3e_rfi_debug=true` to the URL for extra detailed logging - **Check Form Type:** Make sure it's actually a 123FormBuilder form (not a Marketo form or other form type) - **Check iframe:** 123FormBuilder forms often use iframes - make sure the iframe is loaded and accessible

Issue: Duplicate `rfi_submission` events

Solution: - **Check Console Logs:** Enable debug mode and look for messages saying "Skipping hidden conversion form" - this confirms hidden forms are being skipped - **Check Script Assignment:** Verify that 3E_3EI Recruiter Conversion script is handling blind/hidden forms, and 3E_RFI Submit is handling visible forms - **Check for Multiple Scripts:** Make sure you don't have multiple versions of the same script deployed - **Check dataLayer:** After submission, check `dataLayer` - you should see only ONE `rfi_submission` event - **Check**

Form Type: Verify hidden forms have the correct data attributes (`data-mkto-form-purpose="hidden-conversion"`)

Issue: Event not appearing in dataLayer

Solution: - **Check dataLayer Exists:** Open console and type `window.dataLayer` - it should show an array (if undefined, dataLayer doesn't exist) - **Initialize dataLayer:** If dataLayer doesn't exist, add this to your page: `window.dataLayer = window.dataLayer || []` -

Check Console: Enable debug mode and look for "Pushed [event name] event to dataLayer" messages - **Check for Errors:** Look for any JavaScript errors in console that might be

preventing the script from running - **Check GTM Preview:** Use GTM Preview mode to see if events are being detected by GTM - **Check Timing:** Sometimes events are pushed before you check - reload page and check dataLayer immediately after the action

3E_Favicon Injection

File Location: `tags/base-solutions/3E_Favicon Injection.html`

Version: 1.2.1

Type: Custom HTML Tag

Dependencies: 3E Config

What It Does

Dynamically changes the website's favicon (the small icon in the browser tab) based on settings in 3E Config. If no favicon URL is configured, it preserves the existing favicon.

How It Works

A favicon is the small icon that appears in the browser tab next to your website's name. This script can change it to a custom icon:

1. **Page Load:** When the page loads, the script starts
2. **Check Page Type:** It checks if this is a Marketo landing page or a 3enrollment domain (only runs on these pages)
3. **Check Configuration:** It looks to see if you've provided a favicon URL in 3E Config
4. **If Favicon URL Provided:**
5. Removes any existing favicon links from the page
6. Creates new favicon links pointing to your custom icon
7. Adds these links to the page's `<head>` section (where browsers look for favicons)
8. **If No Favicon URL:**

9. Does nothing - leaves the existing favicon alone (won't break anything)

Why This Matters: Custom favicons help with branding - your website tab will show your logo instead of a generic icon.

Validation Process

1. Enable Debug Mode:

2. Set `debugMode: 'true'` in 3E Config

3. Reload page

4. Check Console Logs:

5. Open browser console (F12)

6. Look for: `[3E_Favicon Injection] v1.2.1`

7. Verify "3E Favicon Injection Script Initialized" message

8. Test With Favicon URL Configured:

9. Set `faviconUrl` in 3E Config to a valid image URL

10. Reload page

11. Check browser tab for new favicon

12. Check console for: `[3E Favicon Injection] Favicon injected: [URL]`

13. Inspect page source: Look for `<link rel="icon">` tags in `<head>`

14. Test Without Favicon URL:

15. Remove or empty `faviconUrl` in 3E Config

16. Reload page

17. Check console for: `[3E Favicon Injection] No favicon URL provided - preserving existing favicons`

18. Verify existing favicon remains unchanged

Troubleshooting

Issue: Favicon not changing

Solution: - Verify `faviconUrl` is set in 3E Config - Check that URL is accessible (not broken) - Verify page targeting (must be Marketo landing page or 3enrollment domain) - Check console for injection messages

Issue: Favicon removed when not configured

Solution: - Verify script version is 1.2.1 or later (preserves existing favicons) - Check that `faviconUrl` is empty/not set in 3E Config - Verify console shows "preserving existing favicons" message

Issue: Favicon not showing

Solution: - Verify favicon URL is valid and accessible - Check image format (should be .ico, .png, or .svg) - Clear browser cache and reload - Check browser tab (some browsers cache favicons aggressively)

3E_Sticky Buttons

File Location: `tags/base-solutions/3E_Sticky Buttons.html`

Version: 1.2.4

Type: Custom HTML Tag

Dependencies: 3E Config

What It Does

Creates sticky (always visible) buttons on mobile devices that stay at the bottom of the screen when scrolling. These buttons typically include "Apply Now" and "Request Info" CTAs. On mobile, the buttons stick to the bottom of the viewport. The script also creates a container for the chatbot iframe to be placed in.

How It Works

On mobile devices, this script makes important buttons (like "Apply Now" and "Request Info") stick to the bottom of the screen so they're always visible:

1. **Page Load:** When the page loads, the script starts
2. **Check Page Type:** It checks if this is a Marketo landing page or 3enrollment domain (only runs on these)
3. **Detect Mobile:** It checks if the screen is mobile-sized (less than 576 pixels wide)
4. **Find Buttons:** It looks for the header element that contains your buttons (looks for class `.hdr_right_v2`)
5. **Make Sticky:** On mobile only, it makes the header stick to the bottom of the screen
6. **Add Chatbot Space:** It creates a special container in the sticky header where the chatbot can be placed
7. **Style Buttons:** It adjusts spacing and styling so everything looks good together

Why This Matters: On mobile, users often scroll past important buttons. Making them sticky ensures "Apply Now" and "Request Info" are always visible, which increases conversions.

Validation Process

1. Enable Debug Mode:

2. Set `debugMode: 'true'` in 3E Config

3. Reload page

4. Check Console Logs:

5. Open browser console (F12)

6. Look for: `[3E_Sticky Buttons] v1.2.4`

7. Verify "3E Sticky Buttons Script Initialized" message

8. Test on Mobile Viewport:

9. Resize browser to mobile width (< 576px) or use device emulation

10. Scroll page down

11. Verify buttons stick to bottom of screen

12. Check console for sticky button detection messages

13. Test on Desktop:

14. Resize browser to desktop width (> 576px)

15. Verify buttons do NOT stick (normal positioning)

16. Check console for appropriate messages

17. Verify Chatbot Container:

18. On mobile viewport, inspect page

19. Look for element with ID `chatbot-iframe-container`

20. Verify container is created in sticky header

21. Check console for container creation messages

Troubleshooting

Issue: Buttons not sticking on mobile

Solution: - Verify viewport width is < 576px - Check that header element exists

(`.hdr_right_v2`) - Verify page targeting (must be Marketo landing page or 3enrollment domain) - Check console for sticky button detection messages

Issue: Buttons sticking on desktop

Solution: - Verify viewport width is > 576px - Check that sticky positioning is only applied on mobile - Verify breakpoint logic in script

Issue: Chatbot container not created

Solution: - Verify mobile viewport (< 576px) - Check that sticky header element exists - Verify script version is 1.2.4 or later - Check console for container creation messages

3E_Cloudflare Beacon

File Location: `tags/base-solutions/3E_Cloudflare Beacon.html`

Version: 1.2.0

Type: Custom HTML Tag

Dependencies: 3E Config

What It Does

Integrates Cloudflare Web Analytics tracking on the website. This provides website analytics data through Cloudflare's analytics platform.

How It Works

This script adds Cloudflare Web Analytics tracking to your website:

1. **Page Load:** When the page loads, the script starts
2. **Check Token:** It checks if you've provided a Cloudflare token in 3E Config
3. **If Token Provided:**
 4. Loads Cloudflare's analytics tracking script
 5. Starts tracking with your token
 6. Begins collecting website analytics data
7. **If No Token:**
 8. Script does nothing (skips initialization)
 9. No tracking occurs

Why This Matters: Cloudflare Web Analytics provides website traffic data and insights about how people use your site, similar to Google Analytics but with privacy-focused tracking.

Validation Process

1. Enable Debug Mode:

2. Set `debugMode: 'true'` in 3E Config

3. Reload page

4. Check Console Logs:

5. Open browser console (F12)

6. Look for: `[3E_Cloudflare Beacon] v1.2.0`

7. Verify script initialization message

8. Test With Token Configured:

9. Set `cloudflareToken` in 3E Config

10. Reload page

11. Check console for Cloudflare script loading messages

12. Verify Cloudflare analytics script is loaded (inspect Network tab)

13. Check Cloudflare dashboard for tracking data

14. Test Without Token:

15. Remove or empty `cloudflareToken` in 3E Config

16. Reload page

17. Verify script skips initialization

18. Check console for appropriate messages

Troubleshooting

Issue: Cloudflare tracking not working

Solution: - Verify `cloudflareToken` is set in 3E Config - Check that token is valid and active
- Verify Cloudflare script is loading (check Network tab) - Check Cloudflare dashboard for data

Issue: Script not loading

Solution: - Verify 3E Config is loaded - Check console for initialization messages - Verify no JavaScript errors

Chatbot Solutions

Chatbot solutions provide integration and tracking for chatbot/recruiter functionality. Deploy these when implementing chatbot solutions.

3E_3EI Recruiter Activity

File Location: `tags/chatbot-solutions/3E_3EI Recruiter Activity.html`

Version: 1.1.9

Type: Custom HTML Tag

Dependencies: 3E Config

What It Does

Listens for activity signals from the chatbot iframe and pushes events to the dataLayer. This tracks when users interact with the chatbot (open, respond, click links, close, etc.).

How It Works

This script listens to what's happening in the chatbot and records those events:

1. **Page Load:** When the page loads, the script starts
2. **Set Up Listener:** It creates a listener that watches for messages from the chatbot iframe (the chatbot runs in a separate frame for security)
3. **Security Check:** When it receives a message, it checks where it came from (security check to prevent malicious messages)
4. **Record Events:** When the chatbot does something, it sends a message, and this script records it:
 5. `bot_initiated` - User opened the chatbot
 6. `bot_response` - User sent a message
 7. `bot_click_link` - User clicked a link in the chatbot
 8. `bot_click_app_link` - User clicked an application link
 9. `bot_engagement` - User is actively engaging
 10. `bot_closed` - User closed the chatbot
 11. `bot_email_captured` - User provided their email
12. **Send to dataLayer:** Each event is sent to the dataLayer so other tools can use it

Why This Matters: This tracking helps you understand how people interact with your chatbot - what they ask, what links they click, and whether they provide contact information.

Validation Process

1. Enable Debug Mode:

2. Set `debugMode: 'true'` in 3E Config

3. Reload page with chatbot

4. Check Console Logs:

5. Open browser console (F12)

6. Look for: `[3E_3EI Recruiter Activity] v1.1.9`

7. Verify script initialization message

8. Test Bot Initiated:

9. Open chatbot

10. Check console for: `[3E_3EI Recruiter Activity] bot_initiated event received`

11. Verify dataLayer event: `{event: 'bot_initiated'}`

12. Test Bot Response:

13. Send a message in chatbot

14. Check console for `bot_response` events

15. Verify dataLayer events

16. Test Bot Closed:

17. Close chatbot

18. Check console for: `[3E_3EI Recruiter Activity] bot_closed event received`

19. Verify dataLayer event: `{event: 'bot_closed'}`

Troubleshooting

Issue: No activity events firing

Solution: - Verify chatbot iframe is loaded - Check that chatbot is sending `postMessage` events - Verify origin validation (check console for rejected messages) - Check console for `postMessage` listener setup messages

Issue: Events not appearing in dataLayer

Solution: - Verify dataLayer exists - Check console for event push confirmations - Verify no JavaScript errors

3E_3EI Recruiter Conversion

File Location: `tags/chatbot-solutions/3E_3EI Recruiter Conversion.html`

Version: 2.2.4

Type: Custom HTML Tag

Dependencies: 3E Config, Marketo Forms2

What It Does

Handles chatbot conversion form submissions. When a user submits information through the chatbot (blind form), this script submits the data to Marketo and pushes an `rfi_submission` event to the dataLayer. This is the ONLY script that should handle hidden conversion forms.

How It Works

This script handles form submissions that happen inside the chatbot (called "blind forms" or "hidden conversion forms"):

1. **Page Load:** When the page loads, the script starts
2. **Wait for Forms:** It waits for Marketo's form system to load
3. **Find Hidden Forms:** It looks for hidden forms that are used by the chatbot (these forms are invisible to users but collect their information)
4. **Set Up Handler:** It prepares to handle when someone submits the form through the chatbot
5. **When Form Submits:**
6. Sends the form data to Marketo (saves the lead information)
7. Sends an `rfi_submission` event to the dataLayer (so other tools know a form was submitted)
8. Keeps the user on the same page (doesn't redirect them away)

Important: This is the ONLY script that should handle hidden chatbot forms. The regular 3E_RFI Submit script is told to skip these forms to prevent duplicate submissions.

Why This Matters: When users interact with the chatbot and provide their information, this ensures that data is properly saved to Marketo and tracked, without disrupting the user's

chatbot experience.

Validation Process

1. Enable Debug Mode:

2. Set `debugMode: 'true'` in 3E Config

3. Reload page with chatbot

4. Check Console Logs:

5. Open browser console (F12)

6. Look for: `[3E_3EI Recruiter Conversion] v2.2.4`

7. Verify script initialization message

8. Test Hidden Form Detection:

9. Interact with chatbot to trigger blind form

10. Check console for hidden form detection messages

11. Verify form is marked with data attributes

12. Test Form Submission:

13. Fill out blind form through chatbot

14. Submit form

15. Check console for: `[3E_3EI Recruiter Conversion] Form submitted successfully`

16. Verify dataLayer event: `{event: 'rfi_submission'}`

17. Verify only ONE `rfi_submission` event (not duplicate from 3E_RFI Submit)

18. Verify Marketo Submission:

19. Check Marketo activity log for form submission

20. Verify data was submitted correctly

Troubleshooting

Issue: Hidden form not detected

Solution: - Verify form has data attributes: `data-mkto-form-purpose="hidden-conversion"` or `data-mkto-form-type="hidden-conversion"` - Check that form is in

container with ID `mkto-hidden-form-conversion` - Check console for form detection messages

Issue: Duplicate `rfi_submission` events

Solution: - Verify 3E_RFI Submit is skipping hidden forms (check console logs) - Check that only 3E_3EI Recruiter Conversion handles hidden forms - Verify form is properly marked as hidden conversion form

Issue: Form not submitting to Marketo

Solution: - Verify Marketo Forms2 is loaded - Check Marketo form ID in 3E Config matches form - Check console for submission errors - Verify Marketo API permissions

3E_3EI Recruiter Tracking

File Location: `tags/chatbot-solutions/3E_3EI Recruiter Tracking.html`

Version: 2.1.4

Type: Custom HTML Tag

Dependencies: 3E Config, Marketo Munchkin

What It Does

Sends chatbot events to Marketo via Munchkin tracking. When chatbot events occur (initiated, closed, etc.), this script sends tracking data to Marketo to record the activity.

How It Works

This script takes chatbot activity events and sends them to Marketo for tracking:

1. **Page Load:** When the page loads, the script starts
2. **Wait for Marketo:** It waits for Marketo's tracking system (Munchkin) to be ready
3. **Watch for Events:** It watches the `dataLayer` for chatbot events (like when someone opens the chatbot, sends a message, or closes it)
4. **When Events Happen:**
5. Captures the event information (what happened, when, etc.)
6. Sends this data to Marketo using Munchkin
7. Records the activity in Marketo's system (so you can see chatbot interactions in Marketo)

Why This Matters: This connects chatbot activity to your Marketo lead records, so you can see in Marketo when someone interacted with your chatbot, what they asked about, and whether they provided contact information.

Validation Process

1. Enable Debug Mode:

2. Set `debugMode: 'true'` in 3E Config

3. Reload page with chatbot

4. Check Console Logs:

5. Open browser console (F12)

6. Look for: `[3E_3EI Recruiter Tracking] v2.1.4`

7. Verify script initialization message

8. Test Bot Initiated Tracking:

9. Open chatbot

10. Check console for tracking messages

11. Verify Marketo receives tracking data (check Marketo activity log)

12. Test Bot Closed Tracking:

13. Close chatbot

14. Check console for tracking messages

15. Verify Marketo receives tracking data

Troubleshooting

Issue: Tracking not sending to Marketo

Solution: - Verify Marketo Munchkin is loaded - Check Munchkin ID in 3E Config matches Marketo instance - Verify chatbot events are firing (check dataLayer) - Check console for tracking errors

Issue: Events not appearing in Marketo

Solution: - Verify Munchkin is initialized correctly - Check Marketo activity log for tracking data - Verify event data is being sent - Check for JavaScript errors

3E_3EI Recruiter Unified

File Location: `tags/chatbot-solutions/3E_3EI Recruiter Unified.html`

Version: 1.4.4

Type: Custom HTML Tag

Dependencies: 3E Config

What It Does

Loads the chatbot iframe on the page. It can load either a development or production chatbot based on 3E Config settings. The chatbot appears as a floating widget, and on mobile it can be integrated into the sticky button bar.

How It Works

This script loads and displays the chatbot on your website:

1. **Page Load:** When the page loads, the script starts
2. **Check Settings:** It checks the chatbot environment setting in 3E Config:
3. `disabled` - Chatbot won't appear
4. `dev` - Development chatbot (for testing)
5. `prod` - Production chatbot (live version)
6. **If Disabled:** Script stops and does nothing
7. **If Dev or Prod:**
8. Gets the chatbot ID from 3E Config (which chatbot to load)
9. Creates an iframe (a window that displays the chatbot)
10. Sets the iframe to load the chatbot from the correct URL
11. Places the iframe:
 - On mobile: Inside the sticky button bar (if available)
 - On desktop: In a fixed position (usually bottom-right corner)
12. Sets up communication with the chatbot (so they can send messages back and forth)
13. Handles expanding/collapsing the chatbot when users open or close it

Why This Matters: This is what actually puts the chatbot on your website. Without this script, the chatbot won't appear, even if all the other chatbot scripts are deployed.

Validation Process

1. **Enable Debug Mode:**
2. Set `debugMode: 'true'` in 3E Config
3. Set `chatBotEnvironment: 'prod'` or `'dev'` in 3E Config
4. Set `chatBotProdId` or `chatBotDevId` in 3E Config

5. Reload page

6. Check Console Logs:

7. Open browser console (F12)

8. Look for: `[3E_ChatBot Unified] v1.4.4`

9. Verify chatbot environment and bot ID

10. Test Desktop Chatbot:

11. Verify chatbot iframe appears (usually bottom-right)

12. Check that iframe loads chatbot content

13. Click chatbot to open

14. Verify chatbot expands to full screen

15. Close chatbot

16. Verify chatbot collapses

17. Test Mobile Chatbot:

18. Resize to mobile viewport (< 576px)

19. Verify chatbot appears in sticky button bar

20. Check that chatbot container is created

21. Click chatbot

22. Verify chatbot expands to full screen

23. Close chatbot

24. Verify chatbot collapses to compact view

25. Test Environment Control:

26. Set `chatBotEnvironment: 'disabled'` in 3E Config

27. Reload page

28. Verify chatbot does NOT appear

29. Check console for disabled message

Troubleshooting

Issue: Chatbot not appearing

Solution: - Verify `chatBotEnvironment` is set to 'dev' or 'prod' (not 'disabled') - Check that `chatBotProdId` or `chatBotDevId` is set in 3E Config - Verify bot ID is valid - Check console for initialization errors

Issue: Chatbot not expanding on mobile

Solution: - Verify sticky button container exists - Check that 3E_Sticky Buttons script is deployed - Verify chatbot container is created - Check console for expansion messages

Issue: Chatbot iframe not loading

Solution: - Verify chatbot URL is accessible - Check bot ID is correct - Verify iframe is created (inspect page) - Check console for iframe creation messages

3E_Insights Pixel

File Location: `tags/chatbot-solutions/3E_Insights Pixel.html`

Version: 1.0.7

Type: Custom HTML Tag

Dependencies: 3E Config

What It Does

Loads Marketo Munchkin tracking script and Insights pixel for analytics. This ensures Marketo tracking is available even if Munchkin isn't loaded elsewhere on the page.

How It Works

This script ensures Marketo tracking is available on your page, even if it's not loaded elsewhere:

1. **Page Load:** When the page loads, the script starts
2. **Check for Munchkin:** It checks if Marketo's tracking script (Munchkin) is already loaded on the page
3. **If Not Loaded:**
4. Loads the Munchkin script from Marketo's servers
5. Starts tracking with your Munchkin ID from 3E Config
6. **Load Insights Pixel:** It also loads Marketo's Insights pixel (for additional analytics)
7. **Make Available:** This ensures Marketo tracking is ready for all other tags that need it

Why This Matters: Some websites don't have Marketo tracking loaded by default. This script ensures it's always available, so all your Marketo-related tags can work properly. It's like a safety net - if Munchkin isn't loaded elsewhere, this script loads it.

Validation Process

1. Enable Debug Mode:

2. Set `debugMode: 'true'` in 3E Config

3. Reload page

4. Check Console Logs:

5. Open browser console (F12)

6. Look for: `[3E_Insights Pixel] v1.0.7`

7. Verify script initialization message

8. Test Munchkin Loading:

9. Check Network tab for Munchkin script request

10. Verify Munchkin script loads

11. Check console for Munchkin initialization messages

12. Verify `window.Munchkin` object exists

13. Test Insights Pixel:

14. Check Network tab for Insights pixel request

15. Verify pixel loads correctly

Troubleshooting

Issue: Munchkin not loading

Solution: - Verify `munchkinId` is set in 3E Config - Check that Munchkin ID is valid - Verify Marketo instance URL is correct - Check console for loading errors

Issue: Duplicate Munchkin loading

Solution: - Check if Munchkin is loaded elsewhere on page - Verify script checks for existing Munchkin before loading - Check console for duplicate loading messages

Pop-up Solutions

Pop-up solutions provide pop-up functionality and tracking. Deploy these when implementing pop-up solutions.

3E_Pop-up

File Location: tags/pop-up-solutions/3E_Pop-up.html

Version: 2.3.1

Type: Custom HTML Tag

Dependencies: 3E Config

What It Does

Injects a vendor pop-up snippet (like Pardot, OptinMonster, etc.) into the page. The pop-up code is stored in 3E Config, and this script injects it when enabled.

How It Works

This script adds a pop-up (like a pop-up form or offer) to your website:

1. **Page Load:** When the page loads, the script starts
2. **Check if Enabled:** It checks if pop-ups are enabled in 3E Config (`popupEnvironment: 'enabled'`)
3. **Check for Code:** It checks if you've provided pop-up code in 3E Config (`popupTag`)
4. **If Enabled and Code Exists:**
5. Creates a container on the page
6. Injects your pop-up code into the container
7. Runs any scripts in the pop-up code (so the pop-up actually works)
8. Marks it as injected (so it doesn't add the pop-up twice)
9. **If Disabled or No Code:**
10. Script does nothing (no pop-up appears)

Why This Matters: This allows you to add vendor pop-ups (like Pardot, OptinMonster, etc.) to your website through GTM, without having to modify your website code directly.

Validation Process

1. **Enable Debug Mode:**
2. Set `debugMode: 'true'` in 3E Config
3. Set `popupEnvironment: 'enabled'` in 3E Config
4. Set `popupTag` in 3E Config with pop-up snippet
5. Reload page
6. **Check Console Logs:**

7. Open browser console (F12)
8. Look for: `[3E_Pop-up] v2.3.1`
9. Verify script initialization message

10. **Test Pop-up Injection:**

11. Verify pop-up appears on page
12. Check console for injection messages
13. Inspect page source: Look for pop-up markup
14. Verify pop-up functionality works

15. **Test Disabled State:**

16. Set `popupEnvironment: 'disabled'` in 3E Config
17. Reload page
18. Verify pop-up does NOT appear
19. Check console for disabled message

Troubleshooting

Issue: Pop-up not appearing

Solution: - Verify `popupEnvironment: 'enabled'` in 3E Config - Check that `popupTag` contains valid pop-up snippet - Verify pop-up snippet is correct format - Check console for injection messages

Issue: Pop-up appearing multiple times

Solution: - Verify script version is 2.3.1 or later (has duplicate prevention) - Check that `window.__3E_POPUP_INJECTED__` flag is set - Check console for duplicate prevention messages

Issue: Pop-up scripts not executing

Solution: - Verify pop-up snippet contains valid script tags - Check console for script execution errors - Verify scripts are being extracted and executed correctly

3E_Pop-up Marketo Form

File Location: `tags/pop-up-solutions/3E_Pop-up Marketo Form.html`

Version: 1.2.3

Type: Custom HTML Tag

Dependencies: 3E Config, Marketo Forms2

What It Does

Integrates Marketo forms into pop-ups. When a pop-up displays, it can contain a Marketo form for lead capture.

How It Works

This script makes Marketo forms work properly inside pop-ups:

1. **Page Load:** When the page loads, the script starts
2. **Wait for Forms:** It waits for Marketo's form system to load
3. **Find Pop-up Forms:** It looks for Marketo forms that are inside pop-up containers
4. **Set Up Handling:** It prepares the forms to work correctly within the pop-up (handles display, validation, etc.)
5. **Manage Submission:** It ensures form submissions work properly when submitted from within the pop-up

Why This Matters: Sometimes Marketo forms don't work correctly when placed inside pop-ups. This script ensures they function properly, so users can successfully submit forms from pop-ups.

Validation Process

1. **Enable Debug Mode:**
2. Set `debugMode: 'true'` in 3E Config
3. Reload page with pop-up containing Marketo form
4. **Check Console Logs:**
5. Open browser console (F12)
6. Look for script initialization message
7. Verify Marketo Forms2 detection
8. **Test Form in Pop-up:**
9. Trigger pop-up display
10. Verify Marketo form appears in pop-up

11. Fill out form
12. Submit form
13. Verify form submission works correctly

Troubleshooting

Issue: Form not appearing in pop-up

Solution: - Verify Marketo Forms2 is loaded - Check that form is configured in pop-up - Verify pop-up is displaying correctly - Check console for form detection messages

Issue: Form submission not working

Solution: - Verify Marketo Forms2 is loaded - Check form configuration - Verify form submission handler is set up - Check console for submission errors

3E_Pop-up Tracking

File Location: `tags/pop-up-solutions/3E_Pop-up Tracking.html`

Version: 2.3.2

Type: Custom HTML Tag

Dependencies: 3E Config, Marketo Munchkin

What It Does

Tracks pop-up events (display, close, interaction) and sends tracking data to Marketo via Munchkin. This helps measure pop-up effectiveness and user engagement.

How It Works

This script tracks what happens with pop-ups and sends that data to Marketo:

1. **Page Load:** When the page loads, the script starts
2. **Wait for Marketo:** It waits for Marketo's tracking system (Munchkin) to be ready
3. **Watch for Events:** It watches for pop-up events:
 4. When pop-up is displayed (shown to user)
 5. When pop-up is closed (user dismisses it)
 6. When form in pop-up is submitted
7. **When Events Happen:**
 8. Captures the event information (what happened, when, etc.)
 9. Sends this data to Marketo using Munchkin

10. Records the activity in Marketo (so you can see pop-up interactions in Marketo)

Why This Matters: This helps you measure pop-up effectiveness - how many people see it, how many close it, and how many submit forms. This data appears in Marketo so you can track ROI and engagement.

Validation Process

1. Enable Debug Mode:

2. Set `debugMode: 'true'` in 3E Config

3. Reload page with pop-up

4. Check Console Logs:

5. Open browser console (F12)

6. Look for script initialization message

7. Verify Marketo Munchkin detection

8. Test Pop-up Display Tracking:

9. Trigger pop-up display

10. Check console for tracking messages

11. Verify Marketo receives tracking data

12. Test Pop-up Close Tracking:

13. Close pop-up

14. Check console for tracking messages

15. Verify Marketo receives tracking data

Troubleshooting

Issue: Tracking not sending to Marketo

Solution: - Verify Marketo Munchkin is loaded - Check Munchkin ID in 3E Config - Verify pop-up events are firing - Check console for tracking errors

Issue: Events not appearing in Marketo

Solution: - Verify Munchkin is initialized correctly - Check Marketo activity log for tracking data - Verify event data is being sent - Check for JavaScript errors

General Troubleshooting

Common Issues Across All Tags

Issue: Script Not Loading

Symptoms: - No console logs from script - Functionality not working

Solutions: 1. Verify tag is published in GTM container 2. Check tag trigger conditions (Page View, DOM Ready, etc.) 3. Verify 3E Config variable is set up 4. Check browser console for JavaScript errors 5. Verify tag is not blocked by ad blockers or privacy tools

Issue: 3E Config Not Loading

Symptoms: - Console shows "Failed to load 3E config" errors - Tags not initializing

Solutions: 1. Verify 3E Config variable is created in GTM 2. Check variable name matches exactly (case-sensitive: "3E config") 3. Ensure variable is published 4. Verify variable configuration is valid JSON 5. Check for typos in variable reference: `{{3E config}}`

Issue: Debug Mode Not Working

Symptoms: - No console logs appearing - Can't see what script is doing

Solutions: 1. Verify `debugMode: 'true'` in 3E Config (string 'true', not boolean) 2. Clear browser cache and reload 3. Check browser console is open (F12) 4. Verify no console filters are hiding messages 5. Check that script version supports debug mode

Issue: Duplicate Events

Symptoms: - Same event firing multiple times - DataLayer has duplicate entries

Solutions: 1. Check for duplicate tag deployments in GTM 2. Verify only one script handles each event type 3. Check for hardcoded scripts that might conflict 4. Review tag firing conditions (may be firing multiple times) 5. Use debug mode to identify which script is firing events

Issue: Events Not Appearing in dataLayer

Symptoms: - Script runs but events don't appear in dataLayer - GTM triggers not firing

Solutions: 1. Verify dataLayer exists: `window.dataLayer = window.dataLayer || []` 2. Check console for event push confirmations 3. Verify event format is correct: `{event: 'event_name'}` 4. Check GTM Preview mode to see dataLayer events 5. Verify no JavaScript errors preventing execution

Browser Compatibility Issues

Symptoms: - Script works in some browsers but not others - Console shows compatibility errors

Solutions: 1. Check browser console for specific error messages 2. Verify browser supports required features (ES5+) 3. Test in multiple browsers (Chrome, Firefox, Safari, Edge) 4. Check for polyfills if using newer JavaScript features 5. Verify no browser extensions interfering

Performance Issues

Symptoms: - Page loads slowly - Scripts taking long to initialize

Solutions: 1. Check Network tab for slow script loading 2. Verify scripts are not blocking page render 3. Check for excessive console logging (disable in production) 4. Verify scripts are optimized and minified 5. Check for unnecessary script dependencies

Debug Mode Usage

Enabling Debug Mode

Debug mode provides detailed console logging for all tags. To enable:

1. **In 3E Config Variable:**
2. Set `debugMode: 'true'` (string value, not boolean)
3. Save and publish variable
4. Reload page
5. **For 3E_RFI Submit (Enhanced Debugging):**
6. Add `?3e_rfi_debug=true` to URL
7. This forces debug mode even if 3E Config has it disabled
8. Provides extra logging for form submission detection

What Debug Mode Shows

When debug mode is enabled, you'll see:

- **Script Initialization:**
 - Script name and version
 - Configuration values
- Initialization status
- **Event Tracking:**
 - When events are detected
 - Event data being sent
- dataLayer push confirmations
- **Error Messages:**
 - Detailed error information
 - Stack traces for debugging
- Failed operation details
- **State Changes:**
 - When scripts change state
 - Configuration updates
 - Feature toggles

Console Log Examples

Successful Initialization:

```
[3E_RFI Submit] v1.4.5  
[3E_RFI Submit] Marketo form detected and ready
```

Event Detection:

```
[3E_RFI Submit] Pushed rfi_submission event to dataLayer  
[3E_3EI Recruiter Activity] bot_initiated event received
```

Error Messages:

```
[3E_RFI Submit] Error checking for hidden conversion form: [error details]
[3E Favicon Injection] Failed to load 3E config: [error details]
```

Using Debug Mode for Validation

1. **Enable debug mode** in 3E Config
2. **Open browser console** (F12)
3. **Reload page** and watch for initialization messages
4. **Perform actions** (submit form, interact with chatbot, etc.)
5. **Check console** for expected log messages
6. **Verify events** in dataLayer
7. **Disable debug mode** before going to production

Production Best Practices

- **Always disable debug mode** in production
 - Set `debugMode: 'false'` in 3E Config for production
 - Debug mode adds overhead and exposes internal details
 - Use debug mode only for testing and troubleshooting
-

Additional Resources

Related Documentation

- [tags/README.md](#) - Quick reference for all tags
- [docs/GTM_TAG_UPDATER_SETUP.md](#) - How to update tags across containers
- [docs/GTM_TAG_UPDATER_QUICKSTART.md](#) - Quick start guide for tag updates

Support Contacts

For issues or questions: - Check this guide first - Review tag file headers for version and date information - Check browser console for error messages - Review GTM Preview mode for tag firing information

Document Version: 1.0.0

Last Updated: 2025-12-17

Maintained By: 3E Enrollment Development Team